Appendix B.1

Classes, Objects, Attributes, and Methods

30 minutes instruction + 30 minutes exercises

APPENDIX B.1

Why This Appendix Exists

Students have already used objects — now we name the pattern.

Earlier code already used object behavior:

```
name = "Alice"
name.upper()

numbers = [1, 2, 3]
numbers.append(4)

file_path = Path("data.csv")
file_path.exists()
```

The dot means:
access something belonging to this object

Object
↓
Data associated with it
+
Operations it knows how to perform

APPENDIX B.1

The Familiar Dot Pattern

Strings, lists, dictionaries, and paths all use object.method().

message.strip()	string cleans itself
message.upper()	string changes case
numbers.append(5)	list adds an item
record.get("status")	dict retrieves a value
file_path.exists()	Path checks filesystem

object.method() means behavior is attached to the object

APPENDIX B.1

type() Revisited

The output has been hinting at classes all along.

```
value = "hello"

print(type(value))
```

→

```
<class 'str'>
```

Now the word class matters.

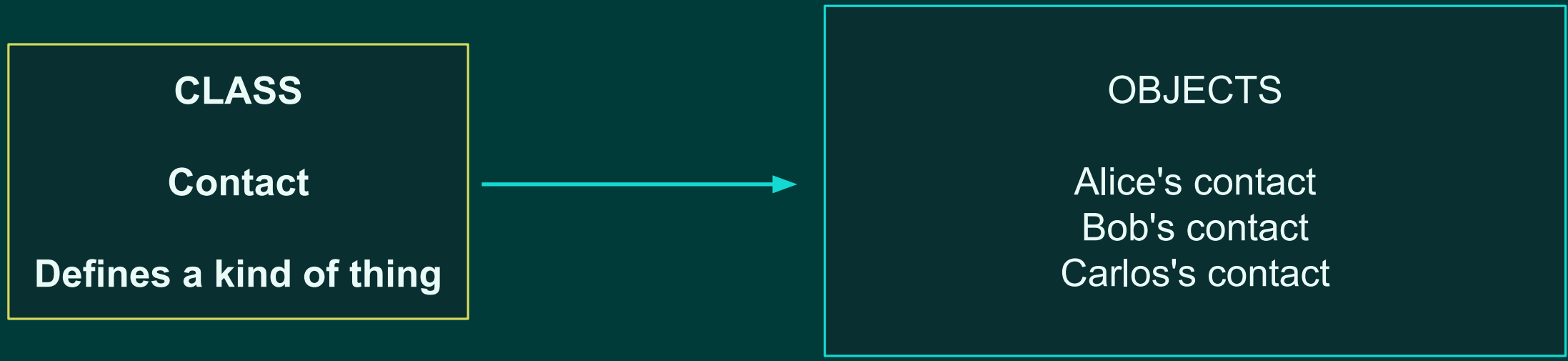
str, int, float, list, dict, set, Path

A class defines a kind of object.

APPENDIX B.1

Class Versus Object

Blueprint versus specific thing created from the blueprint.



Class = definition
Object = one actual instance

APPENDIX B.1

Creating the Smallest Useful Class

Start with the simplest possible custom type.

```
class Contact:
    pass
```

Contact is now a new type your program can create.

```
contact = Contact()

print(type(contact))
```

Defining a class does not create a contact object by itself.

Contact() creates one.

APPENDIX B.1

Adding Data with Attributes

An attribute is data stored on an object.

```
class Contact:
    pass


contact = Contact()

contact.name = "Alice Smith"
contact.email = "alice@example.com"
```

```
print(contact.name)
print(contact.email)
```

object.attribute

contact.name
contact.email

The data belongs to this particular contact object.

APPENDIX B.1

Why __init__ Exists

Objects should begin life with the data they need.

manual setup

```
contact = Contact()
contact.name = "Alice"
contact.email = "alice@example.com"
```

__init__ initializes a newly created object.

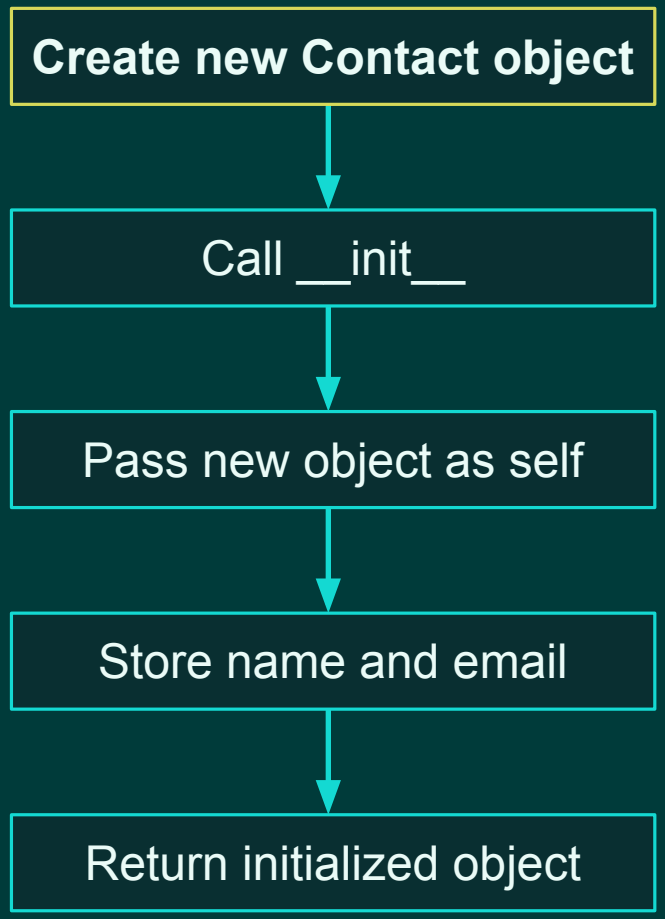
constructor setup

```
class Contact:

    def __init__(self, name, email):
        self.name = name
        self.email = email

contact = Contact(
    "Alice",
    "alice@example.com"
)
```

APPENDIX B.1

__init__ Mental Model

What happens when Python evaluates Contact(...)?




```
contact = Contact(
    "Alice",
    "alice@example.com"
)
```

For this course:
__init__ is the setup step.

APPENDIX B.1

Understanding self

self means the particular object currently being worked with.

```
class Contact:

    def __init__(self, name, email):
        self.name = name
        self.email = email
```

```
alice = Contact("Alice", "alice@example.com")

bob = Contact("Bob", "bob@example.com")
```

During Alice setup:
self refers to alice

During Bob setup:
self refers to bob

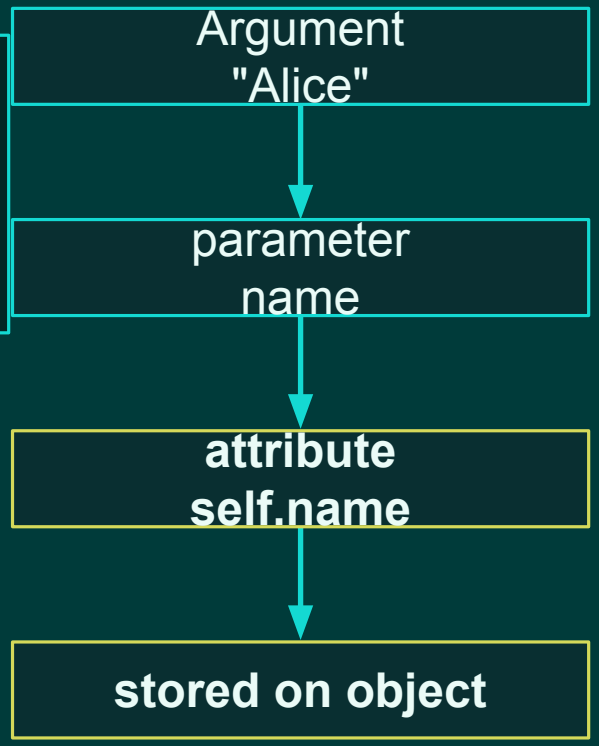
**self is not a magic separate object.
It is the current object.**

APPENDIX B.1

Parameters Versus Attributes

Temporary names become stored object data.

```
def __init__(self, name, email):
    self.name = name
    self.email = email
```



name disappears after the call.
self.name stays on the object.

APPENDIX B.1

Multiple Independent Objects

Each instance has its own attribute values.

```
alice = Contact("Alice", "alice@example.com")
bob = Contact("Bob", "bob@example.com")

print(alice.name)
print(bob.name)
```

alice object
name: Alice
email: alice@example.com

bob object
name: Bob
email: bob@example.com

```
alice.name = "Alice Smith"
```

Changing Alice does not change Bob.

APPENDIX B.1

Adding Behavior with Methods

A function inside a class is usually called a method.

```
class Contact:

    def __init__(self, name, email):
        self.name = name
        self.email = email

    def display(self):
        print(self.name)
        print(self.email)
```

```
contact = Contact("Alice",
"alice@example.com")

contact.display()
```

display() belongs to Contact objects.

APPENDIX B.1

Function Versus Method

Methods are functions associated with objects.

function style

```
display_contact(contact)
```

The contact is passed explicitly.

method style

```
contact.display()
```

The object is still used.
It is received as self.

Conceptually similar — different organization.

APPENDIX B.1

A Method Can Change Object State

State means the current data stored on the object.

```
class Contact:

    def __init__(self, name, email, is_favorite=False):
        self.name = name
        self.email = email
        self.is_favorite = is_favorite

    def mark_as_favorite(self):
        self.is_favorite = True
```

```
contact = Contact("Alice",
"alice@example.com")
contact.mark_as_favorite()

print(contact.is_favorite)  # True
```

The method changed the object.

APPENDIX B.1

Object State + Behavior

A class brings related data and behavior together.

CONTACT OBJECT

state/data
name
email
phone
is_favorite

behavior/methods
mark_as_favorite()
display()

Objects are still normal Python values.

They can be stored in variables, placed in lists, passed to functions, and returned from functions.

data + behavior = one coherent thing

APPENDIX B.1

Dictionary Versus Class

Both are reasonable representations. Classes are not automatically better.

dictionary

```
contact = {
    "name": "Alice",
    "email": "alice@example.com",
    "is_favorite": False,
}

contact["name"]
mark_as_favorite(contact)
```

class object

```
contact = Contact(
    "Alice",
    "alice@example.com"
)

contact.name
contact.mark_as_favorite()
```

Use the representation that makes the program easier to understand.

APPENDIX B.1

When a Class Can Help

Choose classes when data and behavior naturally travel together.

Classes can help when:

- many records share the same fields
- related functions keep receiving the same dictionary
- object behavior has a clear home
- the code reads better as contact.display()

Classes may be unnecessary when:

- the data is simple and temporary
- a dictionary is already clear
- class code adds ceremony
- no behavior belongs with the data

Object-oriented code is a tool, not a requirement.

APPENDIX B.1

Vocabulary Checkpoint

Students should be able to name each part before exercises begin.

```
class Contact:

    def __init__(self, name, email):
        self.name = name
        self.email = email

    def display(self):
        print(self.name)

alice = Contact("Alice", "alice@example.com")
alice.display()
```

Contact
class

alice
object / instance

self.name
attribute

display()
method

self
current object

APPENDIX B.1

Guided Exercise Block

Now build and use a Contact class step by step.

30 minutes

Exercise flow

1. Create class
2. Create objects
3. Add favorite status
4. Add display()
5. Search favorites

Goal: recognize class, object, attribute, method, self, and __init__.

APPENDIX B.1

6 minutes

Exercise 1 — Create a Contact Class

Start with attributes created in __init__.

```
class Contact:

    def __init__(self, name, phone, email):
        self.name = name
        self.phone = phone
        self.email = email
```

Task

Create one Contact object.
Then print name, phone, and email.

```
contact = Contact(
    "Alice Smith",
    "555-123-4567",
    "alice@example.com"
)
```

```
print(contact.name)
print(contact.phone)
print(contact.email)
```

APPENDIX B.1

6 minutes

Exercise 2 — Create Multiple Contacts

A list can contain references to objects.

```
alice = Contact(...)
bob = Contact(...)
carlos = Contact(...)

contacts = [
    alice,
    bob,
    carlos,
]
```

```
for contact in contacts:
    print(contact.name)
```

Same list + loop skills, new item type.

Objects work naturally with variables, lists, loops, and conditions.

APPENDIX B.1

7 minutes

Exercise 3 — Add Favorite Status

Add a default value and a method that changes state.

```
def __init__(
    self,
    name,
    phone,
    email,
    is_favorite=False
):
    self.name = name
    self.phone = phone
    self.email = email
    self.is_favorite = is_favorite
```

```
def mark_as_favorite(self):
    self.is_favorite = True
```

```
alice.mark_as_favorite()

print(alice.is_favorite)
```

Question: what changed — the class or Alice?

APPENDIX B.1

5 minutes

Exercise 4 — Add a Display Method

Put display behavior with the contact object.

```
def display(self):
    print(f"Name: {self.name}")
    print(f"Phone: {self.phone}")
    print(f"Email: {self.email}")
```

Expected result:
Name, phone, and email are printed for Alice.

```
alice.display()
```

Method call reads like a command to the object.

APPENDIX B.1

6 minutes

Exercise 5 — Search Contact Objects

Use conditions with object attributes.

```
contacts = [
    alice,
    bob,
    carlos,
]

for contact in contacts:

    if contact.is_favorite:
        print(contact.name)
```

Task

Mark two contacts as favorite.
Then print only favorite names.

The loop is unchanged.
Only the record shape changed.

APPENDIX B.1

Applied Challenge — Convert Dictionaries to Objects

Move from dictionary records to Contact objects.

starting point

```
contacts = [
    {
        "name": "Alice",
        "email": "alice@example.com",
        "is_favorite": True,
    },
    {
        "name": "Bob",
        "email": "bob@example.com",
        "is_favorite": False,
    },
]
```

target

```
contacts = [
    Contact(
        "Alice",
        "alice@example.com",
        True
    ),
    Contact(
        "Bob",
        "bob@example.com",
        False
    ),
]
```

APPENDIX B.1

15 minutes

Applied Challenge — Add Behavior

Finish the Contact class and explain the vocabulary.

Required methods

```
def mark_as_favorite(self):
    ...

def display(self):
    ...
```

Then explain

```
Contact = class
alice = object / instance
alice.name = attribute
alice.display() = method
self = current Contact object
__init__ = setup step
```

Successful students can read object-oriented code even before mastering every design pattern.

APPENDIX B.1

Exit Check

Object-oriented programming starts with a simple mental model.

Class	Blueprint for a kind of object
Object	One specific instance created from a class
Attribute	Data stored on an object
Method	Behavior attached to an object

Next: recognizing composition, inheritance, and class design tradeoffs.